

Contents

Requirements for authorization in UDM	2
What restrictions do we need?	2
Integration of UDM Authoriztion into the Guardian concept	2
First Alternative (deprecated)	3
Second Alternative	3
Further aspects	5
Extended attributes	5
Conditions provided by UDM	5
Default example roles	5
Wildcard permissions	5
Known problems with Guardian	6
General/Conceptual problems	6
Ambiguity of implementation	6
Capabilities are bound to roles	6
Restricted charset	6
Capability namespace binding restricted to the permission namespace	7
No dynamic contexts allowed	7
No removal in Guardian possible	7
Permission granting: no negative permissions	8
No language to descibe rules, just HTTP API endpoints with JSON payloads	8
Security Impacts	9
Availability of UDM	9
Information disclosure	10
OPA/Rego doesn't know LDAP DNs	10
No real security given by OPA / Guardian: Policy Information endpoint	10
Guardian API design: new/old state of targets not required for filtering	11
Performance Impacts	11
Search results must provide full data to Guardian	11
Guardian API design: extensive data format	11
UDM is synchronous	11
Rule evaluation stays on the client.	11
check-permissions vs get-permissions endpoints	11
UDM actions do a lot of sub-actions	12
Managment API bugs and issues	12
Changing conditions impossible	12
Failed decoding of input JSON data	12
Unreachable API	12
Performance of Management API	12
cyclic dependency problem	13
Questions	13
Conclusion	13
TODOs: descibe the following	13
Comparision with LDAP ACLs	13
Implementation of Authorization in UDM	14
How to initialize a service to enable the authorization engine?	14

Requirements for authorization in UDM

- No LDAP ACL's **strategic decision**
- Authorization for UDM objects types and properties, not LDAP attributes **strategic decision**
- Authorization based on
 - the role (**guardianRoles**) of the actor
 - the `position` of the target
- Probably a future need for “value-based” authorization (e.g. username can not be root, group memberships can not be Domain Admins, certain guardianRoles are allowed)
- Privileged LDAP connection for access to database after authorization (not user connection)
 - This introduces a security risk for UDM extensions. We can not technically enforce that Python code from 3rd parties do the authorization part.
 - * **Product Management decision:** we can ignore this for now, it is the responsibility of the operator and/or the “manufacturer” to ensure that 3rd party extensions work correctly
 - * Furthermore we want to provide the possibility to extend UDM in a declarative way (e.g. via YAML) to get rid of custom Python business logic
- Customers can create roles and assign permissions for those roles
- If possible, the Guardian should be used

Otherwise we don't have clear requirements about the current and future use cases: We currently only have vague statements to support three roles: OU Admin, Computer Join Operator, Helpdesk Operator.

But we have knowledge about the product: * We know the security implications of certain attributes * We know the requirements of the self service * We know the current UCS LDAP ACL's * We know the LDAP ACLs of UCS@school

And with this in mind (and considered), we know that the product should go into the direction that these same things can be realized with UDM permissions

What restrictions do we need?

- no writing to `users/user:guardianRole`, `groups/group:guardianMemberRole`
- no reading of `users/user:password`, `users/user:serviceSpecificPassword` (but maybe write)
- sensitive rules about `users/user:` `overridePWLength`, `overridePWHistory`, `shell`, `unixhome`, `locked/unlock`, `disabled`, `gidNumber` `uidNumber`.
- restrict the possible values for `primaryGroup` and `groups` (don't allow to put into Domain Admins)
- ... to be continued ...

Integration of UDM Authoriztion into the Guardian concept

Guardian is an Authorization Information Point, meaning it doesn't enforce any policies. UDM is the layer which enforces the rule evaluation.

First Alternative (deprecated)

- Permissions for general actions of a UDM module:
 - Permission: `udm:udm:{module}-create` allows to create objects of this module
 - Permission: `udm:udm:{module}-modify` allows to modify objects of this module
 - Permission: `udm:udm:{module}-rename` allows to rename objects of this module
 - Permission: `udm:udm:{module}-remove` allows to remove objects of this module
 - Permission: `udm:udm:{module}-move` allows to move objects of this module
 - Permission: `udm:udm:{module}-search` allows to search objects of this module
 - Permission: `udm:udm:{module}-read` allows to read a specific object of this module
 - Permission: `udm:udm:{module}-reports-type-query` allows to query the available report types of this module
 - Permission: `udm:udm:{module}-report-create` allows to create a report for objects of this module
- For every UDM property of all UDM modules:
 - Permission `udm:udm:{module}-read-property-{property}` allows to read the property value
 - Permission `udm:udm:{module}-search-property-{property}` allows to use the property in a search filter
 - Permission `udm:udm:{module}-write-property-{property}` allows to write the property value in a create or modify action
 - Permission `udm:udm:{module}-readonly-property-{property}` restricts the write permissions back to read permissions again
 - Permission `udm:udm:{module}-none-property-{property}` restricts the write or read permissions back to none permissions again
- Wildcard-Permissions:
 - Permission: `udm:udm:{module}-*` allows any of the above UDM module actions
 - Each of the per-property permissions above allows to specify `*` as property name, which applies to all properties the module provides.
 - This is security critical and then must be combined with further restrictions, for example:
 - * `udm:udm:{module}-write-property-* +`
 - * `udm:udm:{module}-readonly-property-guardianRoles +`
 - * `udm:udm:{module}-none-property-password`
- Advantages:
 - transparent
 - traceable
 - easy to understand
 - only one opaque API: the permission strings are the API
- Disadvantages:
 - makes it necessary to send many permission strings between Guardian and UDM (unless wildcards are used)

Second Alternative

UDM by default provides permissions: For every UDM module:

- Capabilities for general actions of a UDM module:
 - Capability: `condition objectType == "users/user"` grants the permission `udm:udm:create` allows to create objects of this module

- Capability: condition objectType == "users/user" grants the permission udm:udm:modify allows to modify objects of this module
- Capability: condition objectType == "users/user" grants the permission udm:udm:rename allows to rename objects of this module
- Capability: condition objectType == "users/user" grants the permission udm:udm:remove allows to remove objects of this module
- Capability: condition objectType == "users/user" grants the permission udm:udm:move allows to move objects of this module
- Capability: condition objectType == "users/user" grants the permission udm:udm:search allows to search objects of this module
- Capability: condition objectType == "users/user" grants the permission udm:udm:read allows to read a specific object of this module
- Capability: condition objectType == "users/user" grants the permission udm:udm:reports-type-query allows to query the available report types of this module
- Capability: condition objectType == "users/user" grants the permission udm:udm:report-create allows to create a report for objects of this module
- Capability: condition objectType == "users/user" grants the permission udm:udm:* allows any of the above actions
- For every UDM property of this module:
 - * Capability condition objectType == "users/user" and property == \$property grants the permission udm:udm:read-property allows to read the property value
 - * Capability condition objectType == "users/user" and property == \$property grants the permission udm:udm:search-property allows to use the property in a search filter
 - * Capability condition objectType == "users/user" and property == \$property grants the permission udm:udm:write-property allows to write the property value in a create or modify action
 - * Capability condition objectType == "users/user" and property == \$property grants the permission udm:udm:readonly-property restricts the write permissions back to read permissions again
 - * Capability condition objectType == "users/user" and property == \$property grants the permission udm:udm:none-property restricts the write or read permissions back to none permissions again
 - * Wildcard-Permissions:
 - Each of the conditions above allows to specify * (or leave it out!?) as property name, which applies to all properties the module provides.
 - This is security critical and then must be combined with further restrictions, for example:
 - udm:udm:write-property with condition property=* +
 - udm:udm:readonly-property with condition property=guardianRoles‘ +
 - udm:udm:none with condition property=password
- Advantages:
 - smaller data transfer
- Disadvantages:
 - no OR relation possible (as multiple conditions are already bundled with a required AND)
 - capabilities currently can and must be linked to one existing role

- two APIs are opaque to the customer: The permission strings and the capability contents.
- a lot of copy&paste necessary
- not transparent
- harder tracing: it's not visible what UDM module or UDM property is meant
- not easy to understand
- not easy to write

Further aspects

Extended attributes

After the installation of an extended attribute, or an app which provides UDM modules or properties, the rules have to be re-created in Guardian via: `/usr/share/univention-directory-manager "$@" create-permissions`

Conditions provided by UDM

Permissions are granted by Capabilities, which can restrict the permissions by adding certain Conditions.

UDM provides the following conditions: - `udm:conditions:target\_position\_in` with parameters `position=`, `scope=` which restricts the permissions to the given LDAP DN position using one of the LDAP scopes `one`, `base`, `sub`. - `udm:conditions:target\_property\_value` with parameters `property` `operator` `value`, where `operator` is one of: `==`, `!=`, `regex-match`, `regex-nomatch`, `==-i`, `!=i`, `regex-match-i`, `regex-nomatch-i` - `udm:conditions:target\_property\_value` with parameters `property` `operator` `value`, where `operator` is one of: `==`, `!=`, `==-i`, `!=i`, `subtree`, `onelevel`, `subtree-i`, `onelevel-i`. - `udm:conditions:objecttype\_equals` with parameters `objectType=` which restricts the action to any UDM modules name. - `udm:conditions:property\_equals` with parameters `property=` which restricts the action to any UDM modules property name. Must be used with the AND operator and the previous condition. - `(udm:conditions:action\_equals` with parameters `action=` which restricts the action to any of `create`, `modify`, `rename`, `remove`, `move`, `search`, `read`, etc.)

Default example roles

TBD: which ones do we want to expose? UDM provides some default roles, which are using some builtin capabilities: - `udm:udm:domain-users` allows to read everything non-sensible - `udm:udm:domain-admin` allows to write everything - `udm:udm:organizational-unit-admin` allows to be admin of a certain OU given by its context - `udm:udm:helpdesk-operator` - `udm:udm:computer-join-administrator` allows to join computers into the domain, and set their corresponding password - ...

Wildcard permissions

Wildcard permissions allow to not be required to specify a whitelist of all allowed properties to have access to. UDM is a dynamically extensible framework, where new properties can be added 1) via additional schema extensions and 2) on the fly via UDM “extended attributes”.

Supporting wildcard permissions adds a security risk. If customers need to copy our default capability definitions for an example role and in some errata update we introduce new security-relevant properties e.g. another service specific password, customers need to adjust their access

definitions prior to the software upgrade. Otherwise a vulnerability time window exists, where access is allowed to these attributes, even if the customer references (in a currently impossible way) our default policies.

Alternative concept to wildcard permissions would be something like a “permission bundle”, where we group certain properties, e.g. default-users-user-properties, custom-users-user-properties, sensitive-users-user-properties.

Known problems with Guardian

General/Conceptual problems

Ambiguity of implementation

“There should be one – and preferably only one – obvious way to do it.” Zen of Python

Guardian allows several ways to implement a use case and its documentation doesn’t give clear answers how things are supposed to be solved.

One can implement a: `* udm:udm:users-user-{action}` permission, where action is e.g. one of create, modify, remove, rename. `* udm:udm:{action}` permission, which must be used in combination with a condition, checking the `objectType == 'users/user'` `* udm:udm:action` permission, which checks in multiple conditions that `action == 'modify'` and `objectType == 'users/user'`

One can implement a: `* udm:udm:users-user-write-property-description` permission `* udm:udm:write-property-description` permission, which must be used in combination with a condition, checking the `objectType == 'users/user'` and `property == 'description'` `* udm:udm:action` permission, which checks in multiple conditions that `action == 'modify'` and `objectType == 'users/user'` and `property == 'description'`

One can implement a: `* Portal Tile view based on (virtual) roles present in the target` `* Portal Tile view based on the app name in the permission string`

Capabilities are bound to roles

When we want to implement customer use cases in a generic reusable fashion, by providing examples how to realize them, we would create explicit permissions (e.g. `udm:udm:users-user-modify-property`). That would serve as a clear API and allows easy re-use by a customer. If we want to implement generic capabilities, which would serve instead as the API, customers need to copy the whole structure and if we change something in our required examples, they need to adopt it equally to their copies. Capabilities are bound to a specific role. That’s a problem and doesn’t allow re-use but requires hard-copying. To be useable, a capability definition should stand on its own, not yet assigned to any role. Maybe some concept like a capability bundle could enhance this.

Restricted charset

Guardian restricts the charset of permissions (basically: `[a-z0-9]`). UDM modules, properties and extended attributes allow arbitrary characters. We need to create a mapping of UDM module and UDM property names. `* The mapping is irreversible when we need to strip characters (Increases the complexity very much in handling this).` `* The mapping is error prone` `* For`

Administrators this is also intransparent, as they have to remember `pwdChangeNextLogin` and `pwdchangenextlogin`.

1. It's not possible to use `users/user` or `pwdChangeNextLogin`.
2. It would be nice to use the `:` as separator for concepts in permission names e.g. `{app}:{namespace}:$module:$action:$thing`.
3. It would be nice if the whole character set of LDAP DN's is allowed (see later about "Contexts").

→ Guardian could allow all characters except for `&`.

Capability namespace binding restricted to the permission namespace

Guardian has a concept of `app:namespace:thing` for permissions, capabilities, roles, etc. In UDM it would have been nice to use the namespaces for the UDM modules. e.g.: `udm:users/user:...` would allow things like `udm:users/user:{action}:{property}:{detail}`.

But capabilities are bound to be in the same namespace as its permissions (and role?). So it wouldn't be possible to create a capability which grants permissions for more than one UDM module.

No dynamic contexts allowed

While we would benefit from dynamic permissions, because UDM properties and modules can be added dynamically, having such functionality is crucial for Contexts. If we want to use the context Guardian concept we want to extend permissions so operations are e.g. restricted to targets underneath of a certain OU.

1. A context cannot have the full character set of a LDAP DN (basically also restricted to `[a-z0-9]`). So we cannot add a context `udm:udm:ou=admin & ou=ou2,dc=example,dc=org`
2. Every context must be registered in guardian. This requires that everytime a OU is created or removed, the context must be added/removed in Guardian. A listener module would have to do this, which adds unnecessary overall complexity to the whole system.

A concept of named context would be good, so that the value is freely chosen but the context name is registered in guardian. One could name a context `udm:udm:ou` and it's value would be `ou=My school 1,dc=example,dc=org` so that the resulting role string would be `udm:udm:organizational-unit-administrator&udm:udm:ou=ou=My school 1,dc=example,dc=org`.

It would allow be nice if multiple contexts could be given per role, not just only one. This could be realized by separating them by `&`.

→ Values for contexts should be free-form strings, without having to be registered in Guardian.

No removal in Guardian possible

The guardian management API and UI doesn't allow to remove any created object like app, namespace, role, context, permission, condition. Only a capability can be removed.

How are we supposed to evolve our definitions within this?

Permission granting: no negative permissions

Guardian allows to give permissions but has no way to reduce already given permissions by a further capability (additive permission management without subtractive permission management).

This is fundamental for our requirements. UDM is dynamically extensible and is also evolving, so that we sometimes add new properties.

1. Guardian requires for each changeable property a permission to be defined.
2. Guardian doesn't have a wildcard permission granting concept
3. If we introduce a wildcard concept, we cannot use the check-permission endpoint anymore: We have to do the whole rule evaluation in UDM.
4. If we introduce a permission-restriction-afterwards concept we cannot use the check-permission endpoint anymore: We have to do the whole rule evaluation in UDM.
5. If we don't have a wildcard concept, we have to adjust existing rules on every software upgrade / extended attribute.
6. If we don't have a wildcard concept, we will send very large amount of permissions strings on each rule evaluation check.

```
permission_check(  
    general_permission=["users-user:modify"],  
    target_permission=["users-user:write:description"],  
    not_target_permissions=["users-user:readonly:description"],  
    not_permissions=["users-user:none:description"]  
)
```

No language to describe rules, just HTTP API endpoints with JSON payloads

As far as I know, Univention wants to use industry standards, combine existing products, use industry standard software. Univention wants to remove it's own Univention-inventions.

This is not achieved by implementing custom HTTP APIs with custom data formats or just by using OpenPolicyAgent.

The Guardian HTTP APIs introduce Univention inventions, specific data formats in JSON, which must be learned by customers, partners, etc.

Guardian defines the semantic of a language by introducing concepts like “permissions”, “capabilities”, “conditions”, “roles”, “contexts”, etc. Guardian provides no real syntax to realize these concepts. Instead it offers unfinished JSON based HTTP API endpoints with complex data structure. See also Guardian API design: extensive data format.

Other access control systems make their rules describable via a language, such as LDAP ACLs. Guardian allows to describe rules as capabilities, where each capability is like the following JSON format POSTed to the guardian-management-api.

A capability grants permissions to a specific role based on optional conditions:

```
{
  "name": "users-user-creation",  # must be in the same namespace as the role
  "display_name": "Allow the role udm:udm:users-user-creator to create UDM",
  "role": {
    "app_name": "udm",
    "namespace_name": "udm",
    "name": "users-user-creator"
  },
  "relation": "AND",  # operator in which conditions are linked
  "conditions": [
```



```

{
  "app_name": "udm",
  "namespace_name": "conditions",
  "name": "action-equals",
  "parameters": [
    {"name": "action", "value": "create"}
  ]
},
{
  "app_name": "udm",
  "namespace_name": "conditions",
  "name": "objecttype-equals",
  "parameters": [
    {"name": "objectType", "value": "users/user"}
  ]
}
],
"permissions": [
  {
    "app_name": "udm",
    "namespace_name": "udm",
    "name": "create"
  }
]
}

```

This data format is bloated, not changeable, not directly focused on what it offers, not intuitively understandable. Changing the underlying implementation will change everything.

UDM should abstract away these implementation details and provide a easy to understand format, where this HTTP API JSON syntax is created from.

We assume customers don't want to write such error prone JSON blobs. What are configuration formats which are nearly standards, and easy to parse?

URIs are a concept which allow to express this as well. We could also invent a YAML format.

Security Impacts

Availability of UDM

Currently UDM works with no dependencies and only has the dependency on OpenLDAP server availability. Doing authorization via Guardian involves a lot more required components (single points of failure): Keycloak, Guardian Authorization API, Open Policy Agent, PostgreSQL. This opens the way up for:

1. robustness issues
 - if one component is not available, and this can happen often during server maintenance and updates, the central UDM identity management is prevented
 - do we have enough retry mechanisms to deal with this? We has a support case which took weeks to solve because Keycloak couldn't be reached.
2. security issues

- Denial of Service has a larger attack surface (attacking just Keycloak makes UDM unusable)
- Denial of Service of UDM affects the other services. Just throw enough requests against various UDM instances to get the other components Co-DoSed.
- Overtaking one component via one vulnerability in the stack allows overtaking the full domain

3. scalability issues

- the above systems are the security foundation of the domain, i.e. they must not run on member servers or replicas but only on DC Primary and DC backup systems

4. cyclic dependency issues

- e.g. joinscripts
- guardian-management-api → guardian-authorization-api → UDM REST API → guardian-authorization-api

Information disclosure

To prevent information disclosure, the whole implementation must return the same response structure as LDAP would do it. E.g. LDAP says “No Such Object” if a search with a certain search base is done. The `cn=admin` account, with which UDM then runs, has permissions to read all those objects. The UDM interfaces must now raise the same exception and signature. When it would say “Forbidden” instead or the output differs otherwise, an arbitrary user can obtain any information. By combining search bases, search filters and search scopes a lot of attack vectors are possible in every API endpoint and further down, as e.g. syntax classes expose a lot of different possibilities.

This is very tricky to realize, it must be realized everywhere and it must be understood by every further UDM developer so that things don’t break when evolving UDM.

If one measures the statistical average time to get such a “no object” response, one can also differentiate whether an object exists or not. Therefore we also have to do guardian requests for objects, which don’t exist at all. A http request will most likely add enough time to the response so that this gets a real attack vector.

OPA/Rego doesn’t know LDAP DNs

In UDM we use the C library `_ldap` for LDAP DN comparisons to make correct comparisons, as our customer environments use special characters like `+`, `=`, `(`, etc. Depending on the request and where data originates from we have different DN formats. e.g. `uid = foo \+ bar, cn=users` is equal to `uid=foo \2b bar, cn=user`.

So our checks in OPA need to know these normalization rules when comparing DN’s. This is not only from a functional point of view important but also from the security perspective: See the above paragraph; it is required to prevent that actors can enumerate valid LDAP DN’s to check if an object exists, they could just give a different DN representation and get different permission results just because LDAP supports both but guardian wouldn’t.

No real security given by OPA / Guardian: Policy Information endpoint

OPA with Rego is a language to easily write a permission system without side effects (clear input and output) and a restricted environment. That’s great. But in the end, Guardian acts as a Policy Information Endpoint, and the whole permission evaluation is done in the client

(UDM). I think, there is the BSI-Grundschrift requirement, that the rule evaluation can be easily traced in logfiles. Where is this currently possible? I don't see any logging of it.

With an external component involved the debuggability of permission decisions gets harder. Guardian is the Policy Information endpoint, so we have two layers where something is decided.

Guardian API design: new/old state of targets not required for filtering

In the Guardian concept the `target` must always be a dictionary with a `new` and a `old` state. While it's not required, to always set a new state, this depends on the actual conditions. For some operations like filtering the search results, there is no `old` and `new` state - because the state is equal.

Guardian doesn't clearly document, which builtin conditions operate on which state of the target. The whole `new` and `old` states should just be a client issue, as only they and the conditions give meaning to it.

Performance Impacts

Search results must provide full data to Guardian

We must provide full data, all targets e.g. the search results need to provide all properties and all target roles and inherited roles (depending on the dynamic permissions/conditions). This is not a per-se guardian problem but a general one with the allowed flexibility.

Guardian API design: extensive data format

The Guardian authorization & management API data format is very extensive, e.g. instead of sending : imploded strings, it's always a dictionary `{'app\_name': ..., 'namespace\_name': ..., 'permission\_name': ...}`. This requires way more data transfer, serialization and parsing.

Consider, that the whole JSON serialization and de-serialization in 3 components (UDM, Guardian, OPA) will not be negligible.

UDM is synchronous

While the LDAP library supports asynchronicity, our whole UDM code doesn't use it and runs synchronously. This also applies to the guardian rule evaluation.

Rule evaluation stays on the client.

The logic to enforce the permissions is way more code than what Guardian provides. The CPU intensive step stays on client side.

check-permissions vs get-permissions endpoints

Guardian basically only offers two endpoints. To realize our requirements we need them in a combined way. We need to check certain general permissions, some target permissions and receive the whole possible permissions (for reasons see above for wildcard-permissions and afterwards-restrictions). This requires us to do always two requests, which are in the backend doing the same logic but just return different results.

It gets even harder, that we cannot let the permissions checked when we have a search result with a mixed set of objects, for example a search for computer objects will return Domaincontroller Slave and Linux Client objects, with different properties. We cannot specify different permissions we need to check per target. Guardian allows only to check all given permissions for all targets. So we need to make multiple requests.

EDIT: This can (at least partly) be solved by not exposing the module name in a permission, but via a additional check in the condition. This, of course, requires all conditions to be linked with the AND relation. With that, we can send mixed targets to the `get-permission` endpoint and receive specific permissions for each of the target, in one request.

UDM actions do a lot of sub-actions

UDM actions do a lot of sub actions, especially when retrieving objects. We need to check if read permissions for all read references exists so that we don't expose information which is usually not visible to the user.

This requires a lot of checks (via single requests) at different places for one action. UDM is not designed to do a input representation → just store it in LDAP operation. It does a lot of sub-operations. UDM Hooks will be even wilder.

Our use case is a secure implementation, not just a “actor can create a user there” and “actor can receive this user”. And this must be specifiable by an Administrator. And we need to tell the administrator, what secure is and what not, with good demo examples.

Managment API bugs and issues

Changing conditions impossible

It's not possible to change a condition, one just get's an Internal Server Error.

Failed decoding of input JSON data

The management API in my tests crashes permanently randomly. Sometimes the valid JSON input could not be decoded, probably because the full request payload buffer was not read or reading was too early when the request wasn't yet completely transmitted. The reason could be wrong handling of asynchronity.

Unreachable API

Sometimes the Apache gateway says the system is not available: Read timeout: Nothing occurs in the logfiles.

Performance of Management API

Our joinscript took more than 35 minutes to create all the default permission strings for all UDM object properties. Each call takes at least a second. Re-running the joinscript took 45 minutes, as Guardian only allows to either create OR modify a permission.

There is no PUT endpoint, which allows the creation or modification in one step. When creating a permission, which already exists, 100 lines of Traceback are logged.

Simply storing all permissions on local JSON files was done in nearly 1 second. A mass-import of the whole structure would help to reduce the performance costs here and would also make sure we always push a idempotent and consistent state for our app “UDM”.

cyclic dependency problem

The guardian-management-api depends on the guardian-authorization-api, which in turn queries the udm-rest-api, which in turn should query the authorization-api to allow access?

Questions

- How should we solve all the guardian problems in a small time frame, where we have a lot of other issues to solve while we also could just create a simple implementation of all this, which doesn't hinder us in the first step and get us going into a compliant solution?
- How can we implement a solution which allows certain users to bypass the Guardian authorization? It would e.g. help the above cyclic dependency problem and also allows cn=admin, which should just have all rights, to
- Do we need to differentiate permission control for UDM REST API, UMC-UDM and UDM-CLI? Can this be achieved via namespaces or via contexts?
- How to handle situations where one is not allowed to read all the groups of a user (e.g. not the Domain Admins or not the OU2-Teacher group) but he wants to modify the user. The client would send back the received groups and just remove the user from all the other groups not allowed to see.

Conclusion

We don't see the cost-benefit ratio in using the Guardian: * Why should we use it, when we have to adjust so many things * Why should we use it, when it creates much overhead like sending a lot of strings see-saw * If we have to adjust so many things, we have to do in in a generic fashion and have followup work like adjusting manuals, etc. * why should we adjust it in a generic way if we don't know the exact use cases so it gets usefull for everyone * are the specific complex things UDM required usefull for every other guardian user or just for UDM? does it give others any value? * does it make sense to change the Guardian just to satisfy the UDM needs?

TODOs: descibe the following

Comparison with LDAP ACLs

TODO: describe the following from a proplem point perspective

wir müssten bei diesem ganzen ABAC Konzept nochmal das Thema Verzeichnisstruktur bedenken. In LDAP ist es so, dass man um ein Objekt lesen zu können, auf alle übergeordneten Objekte read bzw search-Rechte auf das entry Attribut benötigt und wenn man das Objekt modifizieren will, benötigt man auf das übergeordnete Objekt Lese-Rechte auf das Attribute children. Beispiel: um uid=foo,cn=bar,cn=baz,cn=users,dc=example lesen zu können brauche ich mindestens: read auf uid=foo,cn=bar,cn=baz,cn=users,dc=example attr=entry, objectClass, usw search auf cn=bar,cn=baz,cn=users,dc=example attr=entry (wenn man hier z. B. add auf ein Kindobjekt machen will, dann auch write auf children) search auf cn=baz,cn=users,dc=example attr=entry search auf cn=users,dc=example attr=entry search auf dc=example attr=entry

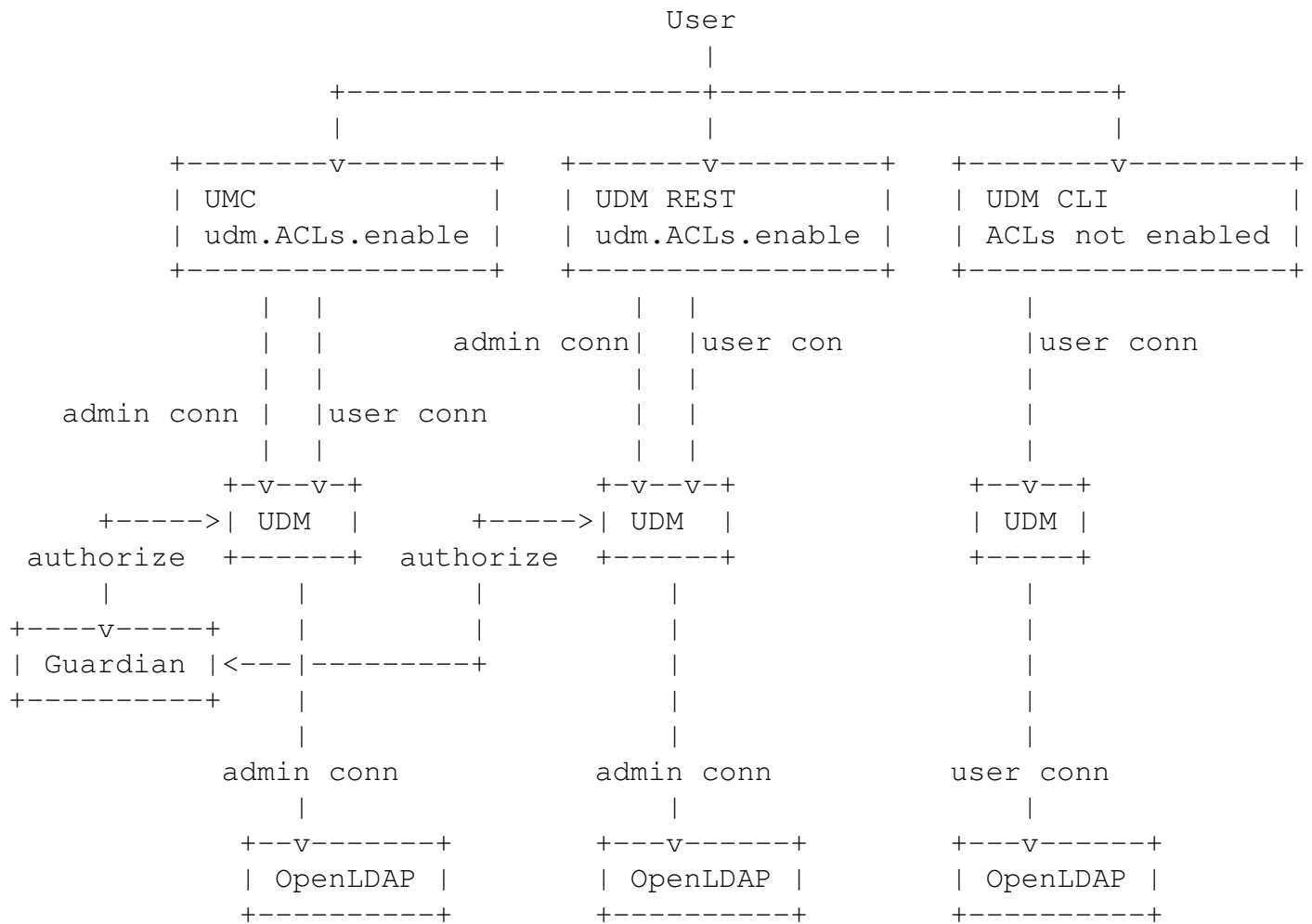
So wie wir es momentan abbilden, ist das völlig außer acht gelassen. Wenn ich uid=foo,cn=bar,cn=baz,cn=users,dc=example lesen kann, gibt es keine Regel, die das einschränkt. Das könnte sehr komische Suchergebnisse liefern, wenn man viele cn's gar

nicht lesen darf aber eine Suche über den LDAP-Baum macht und dann einfach alle Objekte bekommt, die man lesen darf. Dann gibt es viele Parents nicht.

Implementation of Authorization in UDM

Historically authorization in UDM was just realised via LDAP ACL's. A new approach, embedding authorization in UDM has been created.

TODO: adjust diagram to actual state



How to initialize a service to enable the authorization engine?

```

import univention.admin.uldap
from univention.admin.authorization import Authorization

def init_the_service()
    base = ucr.get('ldap/base')
    admin_connection_getter = lambda: univention.admin.uldap.getAdminConnection()
    Authorization.enable(admin_connection_getter)

    # get user connection
    lo = univention.admin.uldap.access(binddn='uid=ouladmin,cn=users,dc=ucs')
    po = position(lo.base)
  
```

```

lo = Authorization.inject_ldap_connection(lo)  # extend user connection

users = modules.get('users/user')
modules.init(lo, po, users)
user = users.object(None, lo, po, )
user.create()

```

This injects the user LDAP connection, so that it provides `lo.authz_connection`, which can be used for certain LDAP operations. In general the regular user connection is passed to every other function call like when receiving other UDM objects.

Only when doing direct LDAP operations, the admin connection has to be used, for example the `simpleLDAP.create()` method uses it like:

```

class simpleLDAP:

    def __init__(self):
        self.authz = Authorization()

    def create(self):
        self.authz.is_create_allowed(self)
        self.lo.authz_connection.add(dn, al)

```

This has to be applied for all operations like `get_schema()`, `add()`, `modify()`, `rename()`, `delete()`, `getPolicies()`. Additionally the methods `get()`, `getAttr()`, `search()` and `searchDn()` must not be used but replaced with equivalents, which filter the results for information the user is allowed to read. It's very important that it raises the same exception signature like LDAP would do, if no permissions exists, otherwise with that information leak it would be possible to find out which objects exists, especially if the user has control over the used LDAP filter, it can be used to obtain arbitrary domain data.

Examples of a filtered search:

1. using a LDAP search, searching for all attributes (assuming the results are only users/user objects!)

```

user_mod = modules.get('users/user')
user = user_mod.object(None, lo, po)

```

```

results = lo.search_filtered({'module': user.module}, user_filter, user_base)

```

2. using a LDAP search with DN as result set (assuming the results are only users/user objects!)

```

user_mod = modules.get('users/user')
user = user_mod.object(None, lo, po)

```

```

results = lo.search_dn_filtered({'module': user.module}, user_filter, user_base)

```